SISTEMA EMPRESARIAL

MONITOREO SÍSMICO, ANÁLISIS PREDICTIVO Y ALERTAS

Plan de desarrollo por fases • Arquitectura modular • React + Python + PostgreSQL

ENFOQUE	OBJETIVO
Monitoreo y visualización	Centralizar eventos sísmicos observados, su ubicación, magnitud, profundidad y evolución temporal.
Machine Learning	Explorar clasificación y regresión para análisis estadístico y estimación experimental de indicadores de riesgo.
Alertas responsables	Distribuir avisos verificables basados en fuentes oficiales y reglas configuradas, con trazabilidad.

Documento de diseño técnico inicial | Versión 1.0 | Octubre de 2026

La propuesta estará organizada para desarrollarse por fases con React + TypeScript, Python para Machine Learning y PostgreSQL como base de datos, separando cada funcionalidad en su propio módulo.

Un aspecto importante del diseño: los sismos no se pueden predecir con fiabilidad indicando exactamente cuándo, dónde y con qué magnitud ocurrirán. Por eso, el sistema diferenciará el monitoreo de eventos reales, la estimación estadística del riesgo, las simulaciones y las alertas oficiales, sin presentar una predicción experimental como una alerta sísmica confirmada.

¿Qué contiene el documento?

- Arquitectura profesional: frontend, API, base de datos, procesos de Machine Learning y despliegue.
- 13 módulos funcionales: dashboard, eventos sísmicos, mapa, análisis estadístico, preparación de datos, modelos ML, estimaciones, alertas, fuentes de datos, reportes, usuarios, auditoría y configuración.
- Campos de cada módulo: identificadores, fechas, magnitud, profundidad, coordenadas, filtros, estados y parámetros.
- PostgreSQL: tablas propuestas, relaciones, índices y reglas de integridad.
- Gráficas de análisis: series temporales, distribución por magnitud, relación entre profundidad y magnitud, mapa geográfico y evaluación de modelos.
- Fases de desarrollo: desde la planificación hasta las pruebas, la seguridad y el despliegue empresarial.
- Machine Learning: preparación de datos, regresión lineal simple y múltiple, clasificación, entrenamiento y evaluación.
- Criterios de aceptación: condiciones para comprobar que cada fase funciona correctamente.
- Mi recomendación: comenzar con la ingesta de datos sísmicos reales, el mapa y el dashboard. Después, incorporar los modelos experimentales y el centro de alertas. Así construiremos una base verificable antes de desarrollar funciones más complejas, manteniendo diferenciados los eventos observados, las estimaciones estadísticas y las alertas oficiales.

1. Resumen ejecutivo

El proyecto propone una plataforma web empresarial para recopilar y consultar eventos sísmicos, analizar patrones históricos, mostrar indicadores geográficos y temporales, ejecutar experimentos de Machine Learning y administrar notificaciones. La interfaz se construirá con React y TypeScript; los servicios de datos y modelos se implementarán con Python mediante una API; PostgreSQL almacenará eventos, usuarios, reglas, predicciones experimentales, auditoría y entregas de alertas.

El sistema no debe anunciar que predice con certeza un terremoto. La ciencia actual no permite predecir de forma fiable y exacta fecha, lugar y magnitud de un sismo. El alcance correcto es monitoreo de eventos, análisis de tendencias, estimaciones probabilísticas claramente etiquetadas, simulaciones y difusión de información oficial. Una alerta temprana, cuando exista una fuente habilitada, se basa en la detección de un sismo ya iniciado y puede ofrecer segundos de aviso, no anticipar el evento antes de que ocurra.

2. Objetivos

- Consolidar eventos sísmicos provenientes de fuentes públicas u oficiales, guardando la fuente y la hora de consulta.
- Mostrar mapas, series temporales, distribución por magnitud/profundidad y filtros por zona y periodo.
- Preparar datos y evaluar modelos supervisados de clasificación y regresión de forma reproducible.
- Configurar alertas operativas con umbrales, zonas, canales, deduplicación, reintentos y registro de entrega.
- Asegurar trazabilidad, control de acceso, privacidad, observabilidad y documentación técnica.
- Permitir crecer por módulos sin acoplar la interfaz, la lógica de negocio y el entrenamiento de modelos.

3. Alcance funcional y límites

Incluido en el alcance	No debe prometerse
Monitoreo de sismos reportados por proveedores de datos; mapa y filtros.	Predecir con certeza la fecha, hora, ubicación y magnitud de futuros terremotos.
Análisis histórico, detección de patrones descriptivos y métricas estadísticas.	Presentar correlaciones o salidas de un modelo experimental como causalidad o certeza.
Estimaciones probabilísticas/indicadores experimentales con incertidumbre y versión del modelo.	Emitir alertas oficiales sin autorización, integración validada o fuente confirmada.
Alertas por eventos recibidos, reglas de negocio y mensajes oficiales verificados.	Sustituir a INDECI, IGP u organismos de protección civil.

4. Arquitectura tecnológica propuesta

Capa	Tecnología	Responsabilidad
Frontend	React + TypeScript + Vite	SPA modular, panel ejecutivo, formularios, mapas, gráficos, accesibilidad y diseño responsive.
UI / gráficos	Tailwind CSS o sistema de diseño; Recharts; Leaflet/MapLibre	Componentes consistentes, gráficas temporales y mapa geográfico.
API de negocio	Python + FastAPI + Pydantic	Autenticación/autorización, validación, endpoints REST, reglas y coordinación de procesos.
Analítica / ML	pandas, NumPy, scikit-learn, joblib	Limpieza, ingeniería de características, entrenamiento, evaluación y registro de modelos.
Persistencia	PostgreSQL	Datos normalizados, índices geográficos/temporales, reglas, auditoría y resultados.
Tareas asíncronas	Celery/RQ + Redis (opcional)	Ingesta programada, procesamiento, notificaciones y tareas largas.
Despliegue	Docker, CI/CD, HTTPS, copias de seguridad	Entornos reproducibles, despliegue controlado, monitoreo y recuperación.

Recomendación: separar el frontend, la API y los procesos de ML en proyectos o paquetes independientes dentro de un monorepositorio. PostgreSQL será la fuente persistente de verdad; los modelos se versionarán como artefactos y metadatos, no como lógica incrustada en React.

5. Estructura de carpetas

```
sismo-platform/
├── apps/
│   └── web/                        # React + TypeScript + Vite
│       └── src/
│           ├── app/                # router, providers, layout, guards
│           ├── modules/
│           │   ├── dashboard/
│           │   ├── eventos/
│           │   ├── mapa-sismico/
│           │   ├── analisis/
│           │   ├── modelos-ml/
│           │   ├── alertas/
│           │   ├── fuentes-datos/
│           │   ├── reportes/
│           │   ├── usuarios-roles/
│           │   └── auditoria/
│           ├── components/         # componentes reutilizables
│           ├── services/           # clientes HTTP por dominio
│           ├── hooks/
│           ├── schemas/            # validación de formularios
│           ├── types/
│           └── lib/
├── services/
│   ├── api/                         # FastAPI, routers, schemas, services
│   ├── ingestion/                   # conectores y sincronización de fuentes
│   └── ml/                          # datasets, features, training, evaluation
├── database/
│   ├── migrations/
│   ├── seeds/
│   └── diagrams/
├── tests/
├── infra/                           # Docker, CI/CD, observabilidad
├── docs/
└── .env.example
```

6. Módulos funcionales y campos

6.1 Inicio / Dashboard

Vista ejecutiva del estado de la plataforma.

Campos principales: Periodo; región/zona; total de eventos; eventos recientes; distribución por magnitud; profundidad media/mediana; última sincronización; estado de fuentes; alertas activas; calidad/latencia de datos.

Interfaz: Tarjetas KPI sobrias, línea de eventos por tiempo, barras por rango de magnitud, mapa de puntos y panel de estado de conectores.

6.2 Eventos sísmicos

Consulta y ficha de eventos recibidos.

Campos principales: ID interno; ID externo; proveedor; fecha/hora UTC; latitud; longitud; magnitud; tipo de magnitud; profundidad_km; región; localidad; estado de revisión; URL fuente; fecha de recepción; hash de origen.

Interfaz: Tabla paginada, filtros por fecha, magnitud, profundidad, proveedor y zona; detalle con mapa y procedencia.

6.3 Mapa sísmico

Explorar espacialmente eventos y zonas.

Campos principales: Área geográfica; latitud/longitud; radio o polígono; rango temporal; magnitud mínima/máxima; capa activa; leyenda; fuente; hora de actualización.

Interfaz: Mapa con agrupación de puntos, leyenda por magnitud, filtros y detalle accesible. No representar como zona de impacto una estimación no validada.

6.4 Análisis estadístico

Describir la historia y la distribución de eventos.

Campos principales: Fecha inicial/final; zona; magnitud mínima; profundidad; agrupación temporal; variable X/Y; resolución; conjunto de datos; método; fecha de ejecución.

Interfaz: Serie temporal, histograma, dispersión magnitud-profundidad, conteo por región y tabla exportable. Mostrar tamaño de muestra y datos faltantes.

6.5 Preparación de datos

Controlar calidad antes de entrenar.

Campos principales: Dataset; fuente; periodo; versión; número de filas; duplicados; nulos por campo; reglas de limpieza; método de imputación; variables; variable objetivo; partición temporal; responsable.

Interfaz: Informe de calidad, vista previa, validaciones, trazabilidad de transformaciones y aprobación del dataset.

6.6 Modelos de Machine Learning

Administrar experimentos de clasificación/regresión.

Campos principales: Nombre; objetivo; algoritmo; versión; dataset_version; features; target; fecha de entrenamiento; hiperparámetros; métricas; periodo de validación; ruta del artefacto; estado; responsable; limitaciones.

Interfaz: Comparación de modelos, métricas, curva real-vs-estimada, matriz de confusión cuando aplique y registro de experimentos.

6.7 Estimaciones experimentales

Publicar resultados exploratorios con contexto.

Campos principales: Modelo; versión; zona/unidad espacial; ventana temporal; indicador estimado; probabilidad o valor; intervalo/incertidumbre; fecha de cálculo; dataset; advertencia; estado de revisión.

Interfaz: Resultados etiquetados como experimentales, con incertidumbre y fecha. No enviar automáticamente como alerta oficial.

6.8 Reglas de alertas

Definir condiciones operativas y destinatarios.

Campos principales: Nombre; tipo de regla; magnitud mínima; profundidad opcional; polígono/zona; fuente permitida; severidad; canal; destinatarios/grupo; horario; estado; cooldown; plantilla; aprobador.

Interfaz: Constructor de reglas, vista de prueba, simulación y activación con permisos.

6.9 Centro de alertas

Rastrear cada alerta desde la detección hasta la entrega.

Campos principales: ID; evento asociado; regla; severidad; título; mensaje; fuente oficial; creado_en; aprobado_por; canal; destinatario; estado de envío; intentos; entregado_en; error; acuse.

Interfaz: Bandeja priorizada, estados pendiente/enviada/fallida/cancelada, reintentos controlados, deduplicación y registro de auditoría.

6.10 Fuentes de datos

Administrar conectores y confiabilidad de la ingesta.

Campos principales: Nombre; organización; URL/API; tipo; frecuencia; última ejecución; último éxito; latencia; estado; límites; licencia; contacto; configuración secreta referenciada.

Interfaz: Estado de conectores, logs, sincronización manual autorizada y métricas de calidad. Nunca guardar secretos en texto plano.

6.11 Reportes

Generar informes operativos y analíticos.

Campos principales: Nombre; periodo; zona; filtros; formato; solicitante; fecha; estado; ruta temporal; expiración.

Interfaz: Exportación CSV/XLSX/PDF según permisos, gráficos con fuente y fecha de corte, reporte de eventos, calidad, modelos y alertas.

6.12 Usuarios y roles

Controlar el acceso por función.

Campos principales: Nombre; correo; rol; estado; último acceso; MFA (si disponible); fecha de alta; permisos asignados.

Interfaz: Roles sugeridos: Administrador, Analista, Operador de alertas, Auditor y Solo lectura.

6.13 Auditoría y configuración

Mantener trazabilidad y parámetros operativos.

Campos principales: Usuario; acción; entidad; ID entidad; antes/después (sin secretos); fecha UTC; IP según política; resultado; correlación; parámetros del sistema.

Interfaz: Historial inmutable o protegido, retención configurable, exportación auditada y control de cambios.

7. Modelo de datos PostgreSQL

Modelo inicial recomendado; adaptar a la fuente de eventos elegida y a los requisitos legales. Usar UUID como claves internas, timestamps en UTC, restricciones CHECK y migraciones versionadas. PostGIS puede añadirse para consultas espaciales.

Tabla	Campos esenciales	Reglas / índices
users	id, full_name, email, status, created_at	email único; no almacenar contraseñas manualmente si se usa un proveedor de identidad.
roles / user_roles	id, name; user_id, role_id	roles normalizados; política de mínimo privilegio.
data_sources	id, name, base_url, source_type, status, last_sync_at, config_ref	secretos fuera de la tabla o cifrados; URL y licencia documentadas.
ingestion_runs	id, source_id, started_at, finished_at, status, records_seen, records_inserted, error_summary	índice por source_id y started_at.
seismic_events	id, external_id, source_id, occurred_at, latitude, longitude, magnitude, magnitude_type, depth_km, place, source_url, received_at, review_status	external_id + source_id único; índices occurred_at, magnitude y coordenadas; validación de lat/lon.
datasets	id, name, version, query_spec, period_start, period_end, row_count, quality_report, created_by, created_at	versionado inmutable de conjuntos publicados.
ml_experiments	id, name, task_type, algorithm, dataset_id, feature_spec, target_name, params, metrics, validation_spec, status, artifact_uri, created_at	guardar semilla, versión de librerías y partición temporal.
model_registry	id, experiment_id, version, status, approved_by, approved_at, limitations	solo modelos aprobados pueden exponerse como estimación publicada.
risk_estimates	id, model_id, area_code, window_start, window_end, metric_name, value, lower_bound, upper_bound, calculated_at, disclaimer	no mezclar con eventos observados; toda salida identifica modelo y versión.
alert_rules	id, name, condition_json, severity, zone_spec, channels, cooldown_seconds, enabled, created_by	validar esquema de condición; permisos para activar.

alerts	id, event_id, rule_id, severity, title, message, source_url, status, created_at, approved_by	deduplicación por evento/regla/canal; historial de estados.
alert_deliveries	id, alert_id, channel, destination_ref, status, attempts, last_attempt_at, delivered_at, error_code	reintentos idempotentes y métricas de entrega.
audit_logs	id, actor_id, action, entity_type, entity_id, before_json, after_json, occurred_at, correlation_id	acceso restringido; no registrar tokens ni secretos.
report_jobs	id, requested_by, report_type, filters_json, status, file_ref, created_at, expires_at	enlaces temporales y acceso autorizado.

8. API REST inicial

Dominio	Endpoints orientativos	Uso
Eventos	GET /api/v1/events; GET /api/v1/events/{id}	Filtros, paginación y detalle de evento.
Dashboard	GET /api/v1/dashboard/summary; GET /api/v1/analytics/timeseries	KPIs y series agregadas.
Mapa	GET /api/v1/events/geojson	Eventos filtrados en formato geográfico.
Fuentes	GET /api/v1/sources; POST /api/v1/sources/{id}/sync	Estado y sincronización protegida.
Datasets	GET/POST /api/v1/datasets; GET /api/v1/datasets/{id}/quality	Versionado y control de calidad.
ML	POST /api/v1/ml/experiments; GET /api/v1/ml/experiments/{id}; POST /api/v1/ml/models/{id}/approve	Entrenamiento asíncrono, métricas y aprobación.
Estimaciones	GET /api/v1/estimates; GET /api/v1/estimates/{id}	Consultar resultados experimentales con disclaimers.
Alertas	GET/POST /api/v1/alert-rules; GET /api/v1/alerts; POST /api/v1/alerts/{id}/approve	Reglas y ciclo de aprobación.
Reportes	POST /api/v1/reports; GET /api/v1/reports/{id}	Generación y descarga autorizada.
Admin	GET /api/v1/audit-logs; CRUD restringido de usuarios/roles	Auditoría y administración.

Estándares: prefijo /api/v1, OpenAPI, respuestas consistentes, códigos HTTP adecuados, paginación, filtros validados, rate limiting, correlation ID y mensajes de error sin información sensible.

9. Fases de desarrollo

Fase 0 — Descubrimiento y viabilidad

Duración estimada: 1 semana. Definir usuarios, zonas iniciales, fuente(s) sísmicas, licencia, frecuencia, volumen, política de alertas y criterios de aceptación. Validar con especialistas el significado de cada indicador.

Entregable: Documento de alcance, riesgos, fuentes elegidas, casos de uso y backlog priorizado.

Fase 1 — Base técnica y diseño UX

Duración estimada: 1 semana. Crear repositorio, React/Vite, FastAPI, PostgreSQL, Docker, variables .env, CI básico, diseño visual, navegación, roles y convenciones.

Entregable: Aplicación base desplegable, estructura modular, esquema inicial y prototipo navegable.

Fase 2 — Identidad, permisos y auditoría

Duración estimada: 1 semana. Autenticación, roles, autorización por endpoint, sesión segura, bitácora de acciones críticas y manejo de errores.

Entregable: Acceso protegido, roles probados y auditoría de operaciones administrativas.

Fase 3 — Ingesta y catálogo de eventos

Duración estimada: 2 semanas. Implementar adaptador para una fuente real, normalización, deduplicación, sincronización incremental, validación geográfica y registro de ejecuciones.

Entregable: Eventos persistidos con procedencia, sincronización monitorizada y pruebas de duplicados.

Fase 4 — Dashboard, mapa y análisis descriptivo

Duración estimada: 2 semanas. KPI, filtros, tabla paginada, detalle, mapa, series temporales, distribución por magnitud/profundidad y exportación básica.

Entregable: Panel útil con gráficos verificables y trazabilidad de los datos mostrados.

Fase 5 — Calidad de datos y datasets

Duración estimada: 1–2 semanas. Reglas de nulos, valores fuera de rango, duplicados, cortes temporales, versionado y reporte de calidad.

Entregable: Datasets versionados, informe de calidad y pipeline reproducible.

Fase 6 — ML supervisado

Duración estimada: 2–3 semanas. Definir una variable objetivo válida, baseline, clasificación/regresión, partición temporal, entrenamiento, métricas y comparación. Evitar fuga de datos y validar con expertos.

Entregable: Experimentos reproducibles y métricas; sin afirmar capacidad de predicción exacta de terremotos.

Fase 7 — Reglas y centro de alertas

Duración estimada: 2 semanas. Configurar umbrales para eventos recibidos, zonas y canales; plantillas, deduplicación, aprobación humana cuando corresponda, reintentos y auditoría.

Entregable: Alertas de prueba y operativas conectadas a fuentes verificadas, con historial de entrega.

Fase 8 — Reportes y administración

Duración estimada: 1–2 semanas. Reportes de eventos, modelos, fuentes y alertas; exportación; configuración; administración de usuarios; retención.

Entregable: Reportes reproducibles y administración con permisos.

Fase 9 — Seguridad, pruebas y despliegue

Duración estimada: 2 semanas. Pruebas unitarias/integración/E2E, carga, recuperación de backup, accesibilidad, revisión de seguridad, monitorización y manuales.

Entregable: Versión candidata, checklist de aceptación, manual técnico y plan de recuperación.

Estimación total orientativa: 14–18 semanas para un MVP robusto con un equipo pequeño y una fuente inicial; puede variar por disponibilidad de datos, integraciones, validación científica y requisitos de seguridad.

10. Machine Learning: aplicación de las operaciones de la hoja

Operación académica	Aplicación en el sistema	Criterio técnico
Describir algoritmos supervisados	Documentar el objetivo, variable objetivo, features y límites de cada experimento.	La tarea se elige según datos y pregunta, no por disponibilidad de un algoritmo.
Definir regresión lineal simple y múltiple	Ejercicio didáctico con variables históricas para estudiar asociaciones numéricas.	No usar una regresión ingenua para afirmar que anticipa un terremoto.
Implementar regresión lineal en Python	Pipeline reproducible en un módulo de laboratorio con datos de entrenamiento y evaluación.	Separar entrenamiento de inferencia; registrar versión, semillas y métricas.
Preparación de datos	Limpieza, detección de duplicados, transformación, codificación y división temporal.	Evitar fuga de información futura y validar calidad de etiquetas.
Entrenamiento del modelo	Entrenar baseline y comparar contra modelos sencillos alternativos.	Conservar dataset y configuración exactos.
Realización de predicciones	Generar estimaciones experimentales claramente etiquetadas y con intervalo/incertidumbre cuando sea posible.	No confundir predicción estadística con aviso sísmico oficial.
Evaluación del modelo	Regresión: MAE, RMSE, R^2; clasificación: precision, recall, F1, matriz de confusión, calibración.	Evaluación temporal, comparación con baseline y revisión de falsos positivos/negativos.

Recomendación científica: antes de elegir un target, acordar con especialistas una pregunta medible, por ejemplo, clasificación de un evento ya reportado por rangos de magnitud o análisis descriptivo de tasas históricas. Un modelo que clasifica registros existentes no demuestra que pueda anticipar futuros terremotos. Las magnitudes, coordenadas y marcas de tiempo deben proceder de fuentes confiables.

11. Gráficas y análisis que debe incluir

Gráfica	Ejes / filtros	Qué permite observar
Eventos a lo largo del tiempo	X: fecha/hora; Y: número de eventos; filtro zona, magnitud y periodo.	Cambios en el volumen de reportes y actividad registrada.
Distribución por magnitud	X: rangos de magnitud; Y: conteo.	Frecuencia relativa por rangos, considerando límites de la fuente.
Magnitud vs. profundidad	X: profundidad km; Y: magnitud; puntos por evento.	Distribución de registros, sin atribuir causalidad.
Mapa geográfico	Longitud/latitud; tamaño/color por magnitud; filtro temporal.	Ubicación de eventos reportados y concentración espacial.
Comparación de modelos	Modelo/versión; MAE/RMSE o precision/recall/F1.	Comparar experimentos bajo la misma partición de evaluación.
Calidad de datos	Porcentaje de nulos, duplicados, fallos por fuente y latencia.	Confiabilidad y frescura de los datos.
Estado de alertas	Alertas por severidad, canal, estado y tiempo de entrega.	Operación, fallos y seguimiento.

Cada gráfica debe mostrar unidad, periodo, filtros aplicados, tamaño de muestra, hora de actualización y fuente. Evitar ejes truncados engañosos y colores que sugieran un nivel oficial de peligro sin una escala validada.

12. Seguridad, confiabilidad y continuidad

- HTTPS obligatorio; secretos mediante variables seguras/secret manager; no incluir credenciales en el repositorio.
- Autorización en backend, validación de entrada, consultas parametrizadas y límites de tamaño/frecuencia.
- CORS restringido, protección frente a abuso, registro de errores sin tokens y política de retención.
- Backups automatizados de PostgreSQL y simulacros de restauración; migraciones revisadas.
- Monitorear disponibilidad de fuentes, retraso de ingesta, fallos de tareas, latencia de API y entregas de alertas.
- Idempotencia para ingesta y envíos; reintentos con backoff; deduplicación por evento, regla y canal.
- Alertas de seguridad/operación separadas de avisos públicos de peligro. Las comunicaciones oficiales deben conservar fuente y enlace.
- Pruebas unitarias, integración, E2E, accesibilidad, carga, permisos y recuperación ante fallos.

13. Diseño profesional de la interfaz

Elemento	Propuesta
Identidad visual	Azul marino para navegación, cyan moderado para acciones y encabezados, blanco y gris claro para superficies.
Layout	Sidebar modular, barra superior con estado de fuentes y usuario, contenido principal con filtros y paneles compactos.
Dashboard	KPI en una sola fila cuando el ancho lo permita; mapa y gráficas con jerarquía clara; evitar exceso de tarjetas.
Tablas	Cabeceras sobrias, filas compactas, búsqueda, filtros, orden, paginación, estados vacíos y carga skeleton.
Estados	Distinguir observado, pendiente de validación, estimación experimental y comunicado oficial con etiquetas y texto explícitos.
Responsive y accesibilidad	Contraste suficiente, navegación por teclado, etiquetas en formularios, mensajes de error y tablas utilizables en pantallas pequeñas.

14. Criterios de aceptación del MVP

- La aplicación puede desplegarse desde instrucciones documentadas y configuración de entorno.
- Los eventos muestran proveedor, hora, ubicación, magnitud, profundidad y enlace de procedencia.
- La ingesta no duplica registros al repetirse y registra cada ejecución y error.

- Dashboard, mapa y gráficas respetan filtros y coinciden con las consultas de la API.
- Los modelos registran dataset, periodo, features, versión, métricas y limitaciones.
- Las estimaciones experimentales no se confunden visual ni semánticamente con alertas oficiales.
- Las reglas de alertas permiten probar condiciones, evitar duplicados y consultar la entrega.
- Los roles bloquean funciones tanto en la interfaz como en la API.
- Los reportes muestran fecha de corte, filtros, fuente y usuario solicitante.
- Existen pruebas, backup probado, guía de operación y procedimiento de incidente.

15. Orden recomendado para comenzar

1. Confirmar alcance del MVP: Perú como primera cobertura, una fuente de eventos y usuarios internos iniciales.
2. Definir la fuente de datos, sus términos de uso, frecuencia y campos disponibles antes de fijar el modelo de datos definitivo.
3. Crear el repositorio y los entornos React, FastAPI y PostgreSQL con Docker y migraciones.
4. Construir primero Eventos + Fuentes + Dashboard con datos reales; no comenzar por el modelo predictivo.
5. Añadir análisis estadístico y control de calidad; luego diseñar una pregunta de ML medible.
6. Implementar alertas operativas con fuentes verificadas, pruebas y controles antes de habilitar mensajes externos.
7. Validar con usuarios y especialistas; recién entonces desplegar el MVP.

16. Glosario

Término	Definición
Evento observado	Registro de un sismo detectado/reportado por una fuente, con sus metadatos y hora de publicación.
Alerta temprana	Aviso emitido después de detectar que un sismo ya comenzó, cuando una infraestructura autorizada lo permite.
Estimación experimental	Resultado estadístico o de ML que requiere validación; no equivale a alerta oficial.
Clasificación	Asignar un registro a una categoría definida, por ejemplo, rangos de magnitud de un evento ya reportado.
Regresión	Estimar una variable numérica; su utilidad depende de una variable objetivo científicamente defendible.
Fuga de datos	Uso accidental de información que no estaría disponible al momento real de la estimación, lo que infla las métricas.
Trazabilidad	Capacidad de saber de qué fuente, dataset, versión, regla y usuario procede cada resultado.

17. Referencias técnicas para validar durante la Fase 0

- Instituto Geofísico del Perú (IGP): revisar canales oficiales de información sísmica y condiciones de uso vigentes.
- INDECI / sistemas oficiales de gestión del riesgo: revisar protocolos y lenguaje autorizado para comunicaciones de emergencia.
- FastAPI: https://fastapi.tiangolo.com/
- React: https://react.dev/
- PostgreSQL: https://www.postgresql.org/docs/
- scikit-learn: https://scikit-learn.org/stable/

Nota: los proveedores, endpoints, licencias y protocolos de emergencia deben confirmarse en la Fase 0. Este documento es una arquitectura inicial, no una certificación científica, un sistema oficial de alerta ni un sustituto de los organismos competentes.